

Simulation framework for Table 1 — fast data-generating process

J. Dedecker, M.-L. Taupin et A.-S. Tocquet

2026-07-01

Contents

1	Objective	1
2	Packages	2
3	Global parameters	2
4	Fast Toeplitz-Gaussian generator	2
5	One Table 1 data set	3
6	Shared metadata	4
7	Example data set	5
8	Monte Carlo framework	6
9	Recommended workflow for a screening study	7

1 Objective

This document implements the continuous, heteroscedastic Table 1 data-generating process with a faster simulator for the correlated Gaussian blocks.

The improvement replaces dense covariance-matrix construction and Cholesky factorization by the exact stationary Gaussian AR(1) recursion

$$X_j = \rho X_{j-1} + \sqrt{1 - \rho^2} \varepsilon_j, \quad \varepsilon_j \stackrel{i.i.d.}{\sim} \mathcal{N}(0, \sigma^2),$$

which gives

$$\text{Cov}(X_j, X_k) = \sigma^2 \rho^{|j-k|}.$$

Thus the simulated covariance is the same Toeplitz covariance as in Table 1, but the large 1982-variable noise block is generated much faster.

2 Packages

```
knitr::opts_chunk$set(echo = TRUE, eval = TRUE)

library(parallel)
```

3 Global parameters

```
set.seed(20260701)

# Table 1 dimensions
p <- 2000L
N <- 500L
n_values <- c(500L, 1000L, 2000L, 2500L)

# Variable groups
n_active <- 10L
n_proxy_and_noise <- 8L
n_toeplitz_noise <- p - n_active - n_proxy_and_noise # 1982

# Toeplitz correlations
rho_active <- 0.57
rho_toeplitz_noise <- 0.70

# Regression coefficients, including the intercept
theta_star <- c(
  -1,
  3.619350,
  -3.274923,
  2.963273,
  -2.681280,
  2, 4, 6, 3, 2, 4
)

stopifnot(length(theta_star) == n_active + 1L)
stopifnot(n_toeplitz_noise == 1982L)
```

4 Fast Toeplitz-Gaussian generator

```
simulate_toeplitz_gaussian_fast <- function(n, d, rho, sigma = 1) {
  stopifnot(d >= 1L, abs(rho) < 1, sigma > 0)

  X <- matrix(0, nrow = n, ncol = d)

  # A stationary AR(1) process starts with marginal variance sigma^2.
  X[, 1L] <- rnorm(n, mean = 0, sd = sigma)
```

```

if (d > 1L) {
  innovation_sd <- sigma * sqrt(1 - rho^2)

  # Draw all innovations at once to reduce random-number-generation overhead.
  innovations <- matrix(
    rnorm(n * (d - 1L), mean = 0, sd = innovation_sd),
    nrow = n,
    ncol = d - 1L
  )

  # Each row is a stationary Gaussian AR(1) sequence.
  for (j in 2:d) {
    X[, j] <- rho * X[, j - 1L] + innovations[, j - 1L]
  }
}

X
}

```

5 One Table 1 data set

This function returns only X and Y by default. It does not create column names or duplicate metadata at every replication, reducing memory allocation in Monte Carlo experiments.

```

simulate_table1_dataset_fast <- function(
  n,
  p = p,
  rho_active = rho_active,
  rho_toeplitz_noise = rho_toeplitz_noise,
  theta_star = theta_star,
  add_names = FALSE
) {
  # -----
  # Group A: X1, ..., X10
  # -----

  X1_to_X5 <- simulate_toeplitz_gaussian_fast(
    n = n,
    d = 5L,
    rho = rho_active,
    sigma = 1
  )

  X6 <- rbinom(n, size = 1L, prob = 0.35)
  X7 <- rchisq(n, df = 2)
  X8 <- X1_to_X5[, 1L] * rpois(n, lambda = 2)
  X9 <- X1_to_X5[, 2L] * rnorm(n, mean = 1, sd = 1)
  X10 <- rt(n, df = 14)

  X_active <- cbind(X1_to_X5, X6, X7, X8, X9, X10)

  # -----

```

```

# Response Y
# -----

linear_predictor <- drop(theta_star[1L] + X_active %*% theta_star[-1L])
epsilon <- X_active[, 5L] * (rchisq(n, df = 3) - 3)
Y <- abs(linear_predictor)^0.8 + epsilon / 3

# -----
# Groups B and C: X11,...,X18
# -----

Z3 <- rnorm(n, mean = 0, sd = 0.40)
Z4 <- rnorm(n, mean = 0, sd = 0.02)
Z5 <- rnorm(n, mean = 0, sd = 0.35)
Z6 <- rnorm(n, mean = 0, sd = 0.55)

X_proxy_and_noise <- cbind(
  X_active[, 1L] + Z3, # X11
  X_active[, 3L] + Z4, # X12
  X_active[, 4L] + Z5, # X13
  X_active[, 5L] + Z6, # X14
  Z3, Z4, Z5, Z6      # X15,...,X18
)

# -----
# Group D: X19,...,X2000
# -----

X19_to_X2000 <- simulateToeplitzGaussianFast(
  n = n,
  d = p - 18L,
  rho = rhoToeplitzNoise,
  sigma = 1
)

X <- cbind(X_active, X_proxy_and_noise, X19_to_X2000)

stopifnot(ncol(X) == p)
stopifnot(length(Y) == n)

if (add_names) {
  colnames(X) <- paste0("X", seq_len(p))
}

list(X = X, Y = Y)
}

```

6 Shared metadata

Keep this object once, outside Monte Carlo replications.

```
table1_metadata <- list(
  p = p,
  N = N,
  n_values = n_values,
  theta_star = theta_star,
  rho_active = rho_active,
  rho_toeplitz_noise = rho_toeplitz_noise,
  active_indices = 1:10,
  proxy_indices = 11:14,
  null_noise_indices = 15:18,
  toeplitz_noise_indices = 19:p,
  table1_positive_indices = 1:14,
  table1_null_indices = 15:p
)
```

7 Example data set

```
example_dataset <- simulate_table1_dataset_fast(
  n = 500L,
  p = p,
  rho_active = rho_active,
  rho_toeplitz_noise = rho_toeplitz_noise,
  theta_star = theta_star,
  add_names = FALSE
)

dim(example_dataset$X)
```

```
## [1] 500 2000
```

```
head(example_dataset$X[, 1:12])
```

```
##
## [1,] -1.4355380 -0.745965252 -0.3993645 -1.0974246 -1.346086 0 1.1664121
## [2,] -0.1176663 0.518621264 -0.1186191 0.4066586 0.594730 0 1.3596610
## [3,] -0.4251678 -0.758236177 -1.3911230 -0.6939059 1.271191 1 1.4013377
## [4,] -1.6481772 0.439768124 0.6218423 -1.3660013 -1.710600 0 0.2655433
## [5,] 1.2953495 -0.002581404 2.2393394 2.9274772 2.105675 0 4.2149682
## [6,] 2.4469903 1.612345458 1.7068616 2.4042360 2.129690 0 0.0412528
##
## X8 X9 X10
## [1,] -2.8710760 -2.521103597 0.36003088 -1.6071011 -0.37930167
## [2,] -0.5883314 0.975104042 -0.07709902 -0.3164378 -0.09814359
## [3,] -0.8503356 -0.534313357 1.14075563 -0.2027783 -1.40137392
## [4,] -8.2408860 0.730615323 0.15157799 -0.8076011 0.60711880
## [5,] 2.5906989 -0.002566006 0.33368669 0.5259833 2.24192690
## [6,] 2.4469903 2.329555139 -0.61526796 2.3283051 1.70554012
```

```
head(example_dataset$Y)
```

```
## [1] 6.792479 3.551713 7.321862 14.414294 19.436390 7.457500
```

```
# Empirical correlation of X1,...,X5, to inspect the rho = 0.57 block:  
round(cor(example_dataset$X[, 1:5]), 3)
```

```
##  
## 1.000 0.561 0.266 0.143 0.073  
## 0.561 1.000 0.549 0.272 0.142  
## 0.266 0.549 1.000 0.590 0.331  
## 0.143 0.272 0.590 1.000 0.560  
## 0.073 0.142 0.331 0.560 1.000
```

```
# Empirical correlation of X150,...,X155, to inspect the rho = 0.7 block:  
round(cor(example_dataset$X[, 150:155]), 3)
```

```
##  
## 1.000 0.714 0.480 0.328 0.214 0.144  
## 0.714 1.000 0.704 0.470 0.307 0.232  
## 0.480 0.704 1.000 0.675 0.441 0.317  
## 0.328 0.470 0.675 1.000 0.715 0.470  
## 0.214 0.307 0.441 0.715 1.000 0.670  
## 0.144 0.232 0.317 0.470 0.670 1.000
```

8 Monte Carlo framework

Use the following only when you need all simulated data sets. It is memory intensive for large n and large N .

```
simulate_table1_monte_carlo_fast <- function(  
  n,  
  N = N,  
  mc.cores = 1L  
) {  
  one_draw <- function(i) {  
    simulate_table1_dataset_fast(n = n, add_names = FALSE)  
  }  
  
  if (.Platform$OS.type == "windows" || mc.cores <= 1L) {  
    lapply(seq_len(N), one_draw)  
  } else {  
    parallel::mclapply(  
      X = seq_len(N),  
      FUN = one_draw,  
      mc.cores = mc.cores,  
      mc.set.seed = TRUE,  
      mc.preschedule = FALSE  
    )  
  }  
}
```

9 Recommended workflow for a screening study

When screening will immediately follow data generation, do not store all X and Y matrices. Compute the screening output inside each replication and return only the compact statistic or selected indices.

```
one_draw_returning_summary <- function(i, n) {  
  dat <- simulate_table1_dataset_fast(n = n, add_names = FALSE)  
  
  # Replace this placeholder with the desired screening procedure.  
  # result <- screening_test_matrix(dat$X, dat$Y, q = 0.10)  
  
  # Return only compact output, for example:  
  # list(selected_indices = result$selected_indices)  
}  
  
n_cores <- min(8L, parallel::detectCores(logical = FALSE))  
  
# summaries <- parallel::mclapply(  
#   seq_len(N),  
#   one_draw_returning_summary,  
#   n = 500L,  
#   mc.cores = n_cores,  
#   mc.set.seed = TRUE,  
#   mc.preschedule = FALSE  
# )
```